

03_relacion-alertas-frontend-flujo-soc

#python

#api

#fastapi

#swagger

#pydantic

#PostgreSQL

#SQLAlchemy

#backend

#frontend

#SIEM

#SOC

1 Objetivo de la nota

Esta nota explica cómo se relaciona la gestión de alertas del backend con el flujo visual del frontend y con una lógica básica de trabajo SOC.

A diferencia de las notas anteriores, esta no analiza un único archivo línea por línea. Su objetivo es unir los conceptos ya estudiados:

```
Event
Rule
Alert
AlertOut
AlertUIOut
alerts.py
frontend
flujo SOC
```

El objetivo principal es entender cómo una alerta pasa de ser un registro generado automáticamente por el backend a convertirse en un elemento consultable y gestionable por el usuario.

2 Archivos relacionados

Los archivos principales relacionados con esta nota son:

```
backend/app/api/routes/alerts.py
backend/app/schemas/alert.py
backend/app/models/alert.py
backend/app/models/rule.py
backend/app/models/event.py
backend/app/api/routes/ingest.py
```

También se relaciona con la parte frontend del proyecto:

```
frontend/alert.html
frontend/assets/alerts.js
frontend/assets/alert_detail.js
frontend/assets/app.js
frontend/assets/styles.css
```

Relación general:

```
ingest.py
↓
genera alertas

alerts.py
↓
expone endpoints de consulta y actualización

schemas/alert.py
↓
define respuestas básicas y enriquecidas

frontend
↓
consume endpoints /alerts y /alerts/ui
```

3 Posición de las alertas dentro del flujo SIEM

El flujo general del laboratorio es:

1. Se crea una regla.
2. Se ingesta un evento.
3. El evento se evalúa contra reglas activas.
4. Si una regla coincide, se genera una alerta.
5. La alerta se consulta desde API o frontend.
6. El usuario puede cambiar su estado.

Representado como cadena técnica:

```
POST /rules
  ↓
Rule

POST /ingest
  ↓
Event
  ↓
Rule + Event
  ↓
Alert

GET /alerts/ui
  ↓
frontend

PATCH /alerts/{alert_id}
  ↓
cambio de estado
```

La alerta es el punto donde el laboratorio deja de ser solo un sistema de almacenamiento de eventos y pasa a representar una lógica básica de detección y gestión.

4 Generación de alertas

Las alertas no se crean manualmente desde `alerts.py`.

Se generan en:

```
backend/app/api/routes/ingest.py
```

Cuando una regla coincide con un evento, el backend crea un objeto `Alert` :

```
alert = Alert(
    rule_id=rule.id,
    event_id=ev.id,
    title=f"Rule matched: {rule.name}",
    group_key=group_key,
)
```

Esto significa que una alerta nace con tres relaciones importantes:

```
rule_id
  ↓
indica qué regla se ha disparado

event_id
  ↓
indica qué evento ha provocado la alerta
```

```
group_key
  ↓
permite agrupar alertas por host u otra entidad
```

La relación base es:

```
Rule + Event → Alert
```

5 Modelo mental de una alerta

Una alerta no es simplemente un mensaje.

En este proyecto, una alerta representa una detección con contexto.

Una alerta contiene:

```
id
rule_id
event_id
title
group_key
status
created_at
updated_at
```

Pero para que sea útil en una interfaz, normalmente también se necesita saber:

```
nombre de la regla
timestamp del evento
origen del evento
severidad del evento
mensaje del evento
```

Por eso existen dos tipos de respuesta:

```
AlertOut
AlertUIOut
```

6 Diferencia entre AlertOut y AlertUIOut

El schema básico es:

```
AlertOut
```

Sirve para representar la alerta tal como está en la tabla `alerts`.

Incluye:

```
id
rule_id
event_id
title
group_key
status
created_at
updated_at
```

El schema enriquecido es:

```
AlertUIOut
```

Hereda de `AlertOut` y añade:

```
rule_name
event_ts
event_source
event_severity
event_message
```

La relación es:

```
AlertOut
  ↓
AlertUIOut
  ↓
alerta preparada para frontend
```

Esto permite que la interfaz no tenga que consultar por separado:

```
/alerts
/rules
/events
```

En su lugar, puede consumir directamente:

```
GET /alerts/ui
```

7 Por qué existe `/alerts/ui`

El endpoint:

```
GET /alerts/ui
```

está pensado para devolver alertas ya preparadas para una tabla o vista de frontend.

En lugar de devolver solo IDs, devuelve contexto legible.

Ejemplo conceptual de respuesta:

```
{
  "id": 1,
  "rule_id": 2,
  "event_id": 15,
  "title": "Rule matched: Failed login auth",
  "group_key": "server-01",
  "status": "open",
  "created_at": "2026-01-15T12:00:00",
  "updated_at": "2026-01-15T12:00:00",
  "rule_name": "Failed login auth",
  "event_ts": "2026-01-15T11:59:58",
  "event_source": "auth",
  "event_severity": 4,
  "event_message": "Failed login attempt for user admin"
}
```

Esto permite que el frontend muestre información útil como:

```
Regla: Failed login auth
Origen: auth
Severidad: 4
Mensaje: Failed login attempt for user admin
Estado: open
```

Sin hacer consultas adicionales.

8 Relación entre `/alerts/ui` y los joins

Para construir `AlertUIOut`, el backend hace joins entre:

```
alerts
rules
events
```

La consulta selecciona:

```
Alert
Rule.name
Event.ts
Event.source
Event.severity
Event.message
```

Relación SQL conceptual:

```
alerts.rule_id = rules.id
alerts.event_id = events.id
```

Representado visualmente:

```
alerts
├ rule_id  ───> rules.id
└ event_id ───> events.id
```

Esto permite transformar una alerta técnica en una alerta entendible.

Sin el join, el frontend vería solo:

```
rule_id = 2
event_id = 15
```

Con el join, puede mostrar:

```
rule_name = "Failed login auth"
event_source = "auth"
event_message = "Failed login attempt for user admin"
```

9 Flujo frontend esperado

Aunque el frontend se analizará en otro módulo, el flujo esperado con alertas sería:

1. El usuario abre la página de alertas.
2. El frontend llama a `GET /alerts/ui`.
3. El backend devuelve alertas enriquecidas.
4. El frontend pinta una tabla o listado.
5. El usuario revisa una alerta.
6. El usuario puede abrir el detalle.
7. El frontend llama a `GET /alerts/{alert_id}/ui`.
8. El usuario puede cambiar el estado.
9. El frontend llama a `PATCH /alerts/{alert_id}`.

Diagrama:

```
Frontend alerts.js
  ↓
GET /alerts/ui
  ↓
tabla de alertas

Frontend alert_detail.js
  ↓
GET /alerts/{id}/ui
  ↓
detalle de alerta

Botón de estado
  ↓
PATCH /alerts/{id}
  ↓
open / ack / closed
```

10 Ciclo de vida SOC de una alerta

El proyecto define tres estados:

```
open
ack
closed
```

Estos estados representan un flujo SOC simplificado.

open

Significa que la alerta está abierta o pendiente de revisión.

Es el estado inicial por defecto.

En el modelo `Alert`, el estado tiene:

```
server_default = "open"
```

Por tanto, una alerta nueva nace como `open`.

ack

Significa que la alerta ha sido reconocida.

En un SOC real, esto suele indicar que un analista ya la ha visto o la está investigando.

No significa que esté resuelta.

Representa un punto intermedio:

```
open → ack
```

closed

Significa que la alerta se ha cerrado.

Puede interpretarse como:

```
resuelta
descartada
falso positivo revisado
incidente gestionado
```

En este MVP no se guarda motivo de cierre, pero el estado permite distinguir alertas pendientes de alertas ya tratadas.

Flujo completo:

```
open → ack → closed
```

1.1 Relación entre estado y anti-duplicado

El estado de una alerta no solo afecta a la interfaz.

También afecta al motor de reglas.

En `ingest.py`, antes de crear una nueva alerta, el sistema busca si ya existe una alerta activa para la misma regla y grupo.

Los estados considerados activos son:

```
open
ack
```

Esto significa:

```
Si ya existe una alerta open o ack:
    no crear otra alerta duplicada

Si la alerta está closed:
    se permite crear una nueva alerta futura
```

Representación:

```
open
  ↓
bloquea duplicados

ack
  ↓
bloquea duplicados

closed
  ↓
no bloquea duplicados
```

Esto hace que el estado tenga impacto operativo.

Cerrar una alerta no solo cambia su visualización: también permite que el sistema vuelva a alertar si vuelve a ocurrir el patrón.

1.2 Importancia de `group_key`

El `group_key` es clave en el flujo de alertas.

Se calcula durante la ingesta a partir de:

```
meta.host
```

Ejemplo:

```
{
  "meta": {
    "host": "server-01"
  }
}
```

Resultado:

```
group_key = "server-01"
```

Este valor permite agrupar alertas por entidad.

En un contexto SOC, esto puede representar:

```
host afectado
usuario
IP
servicio
```

En este MVP se usa principalmente `host`.

Relación:

```
Event.meta.host
      ↓
group_key
      ↓
Alert.group_key
      ↓
filtros / duplicados / threshold / throttle
```

13 Filtros útiles para el frontend

El endpoint `/alerts/ui` permite varios filtros:

```
status
group_key
rule_id
severity_min
severity_max
source
q
limit
offset
```

Estos filtros permiten construir una interfaz más útil.

Filtrar por estado

```
GET /alerts/ui?status=open
```

Permite ver solo alertas abiertas.

Uso típico:

```
bandeja de alertas pendientes
```

Filtrar por grupo

```
GET /alerts/ui?group_key=server-01
```

Permite ver alertas de un host concreto.

Uso típico:

```
investigar actividad asociada a una máquina
```

Filtrar por regla

```
GET /alerts/ui?rule_id=3
```

Permite revisar alertas generadas por una regla concreta.

Uso típico:

```
analizar qué regla genera más ruido
```

Filtrar por severidad

```
GET /alerts/ui?severity_min=4
```

Permite mostrar solo alertas cuyo evento asociado tenga severidad alta.

Uso típico:

```
priorizar alertas más críticas
```

Filtrar por origen

```
GET /alerts/ui?source=auth
```

Permite mostrar alertas asociadas a eventos de un origen concreto.

Uso típico:

```
ver alertas de autenticación
```

Buscar texto

```
GET /alerts/ui?q=login
```

Busca en:

```
Alert.title  
Event.message
```

Uso típico:

```
buscar alertas relacionadas con login, blocked, denied, admin...
```

14 Relación entre `/alerts/ui` y `/alerts/ui/count`

Para construir una interfaz paginada, no basta con pedir los datos.

También conviene saber cuántos resultados totales existen.

Por eso existe:

```
GET /alerts/ui/count
```

Ejemplo de flujo:

```
GET /alerts/ui?status=open&limit=50&offset=0  
GET /alerts/ui/count?status=open
```

La primera llamada devuelve los primeros 50 resultados.

La segunda devuelve el total de alertas abiertas.

Esto permite mostrar en frontend:

```
Mostrando 50 de 237 alertas abiertas
```

La clave es que ambos endpoints usan la misma función de filtros:

```
_apply_ui_filters
```

Así se mantiene coherencia entre listado y conteo.

1.5 Pantalla de detalle de alerta

Para una vista de detalle, el endpoint más útil es:

```
GET /alerts/{alert_id}/ui
```

Este endpoint devuelve una alerta enriquecida concreta.

Permite mostrar:

```
Título de la alerta
Estado
Fecha de creación
Fecha de actualización
Nombre de la regla
Origen del evento
Severidad
Mensaje del evento
```

Esto puede alimentar una página como:

```
frontend/alert.html
```

o un script como:

```
frontend/assets/alert_detail.js
```

La idea es que el usuario no vea solo un ID técnico, sino el contexto completo.

1.6 Actualización desde frontend

El endpoint:

```
PATCH /alerts/{alert_id}
```

permite cambiar el estado.

Payload esperado:

```
{
  "status": "ack"
}
```

o:

```
{
  "status": "closed"
}
```

```
}
```

Flujo:

```
Usuario pulsa botón
  ↓
frontend envía PATCH
  ↓
backend valida AlertUpdate
  ↓
actualiza Alert.status
  ↓
commit
  ↓
devuelve AlertOut actualizado
```

Esto permite implementar botones como:

```
Reconocer
Cerrar
Reabrir
```

Reabrir sería volver a poner:

```
status = open
```

si la interfaz lo permite.

17 Relación con métricas

Las alertas también alimentan las métricas del sistema.

El endpoint `/metrics` utiliza la tabla `alerts` para calcular:

```
alerts_total
alerts_by_status
alerts_by_group_key_top
```

Por tanto, cuando el usuario cambia estados:

```
open → ack
ack → closed
```

las métricas por estado cambian.

Ejemplo:

```
Antes:
open = 10
ack = 2
closed = 5

Después de cerrar una alerta:
open = 9
ack = 2
closed = 6
```

Esto permite que el dashboard refleje el estado operativo del laboratorio.

18 Relación con el flujo SOC

Un flujo SOC básico puede representarse así:

1. El sistema genera una alerta.
2. La alerta aparece como open.
3. El analista revisa el contexto.
4. Si decide investigarla, la marca como ack.
5. Si la resuelve o descarta, la marca como closed.
6. Las métricas reflejan el cambio.
7. El motor de reglas puede volver a alertar si se repite el patrón y la anterior está closed.

Representación:

```
Detección
  ↓
open
  ↓
ack
  ↓
closed
```

Aunque es simple, este ciclo ya representa una lógica operativa bastante realista para un MVP.

19 Qué hace y qué no hace este módulo

Sí hace

```
Consulta alertas.
Filtra alertas.
Enriquece alertas con datos de regla y evento.
Cuenta alertas filtradas.
Consulta detalles.
Actualiza estado.
Permite un flujo básico open / ack / closed.
```

No hace

```
No crea alertas manualmente.
No borra alertas.
No asigna alertas a usuarios.
No guarda comentarios de investigación.
No calcula severidad propia de alerta.
No implementa SLA.
No implementa historial de cambios.
```

Estas ausencias son normales en un MVP.

El módulo está centrado en una gestión básica, clara y demostrable.

20 Decisiones técnicas importantes

Separar AlertOut y AlertUIOut

Esto permite tener dos niveles de respuesta:

```
AlertOut
  ↓
mínimo y directo

AlertUIOut
  ↓
enriquecido para interfaz
```

Buena decisión porque evita sobrecargar siempre las respuestas simples.

Reutilizar `_apply_ui_filters`

Evita duplicar lógica entre:

```
GET /alerts/ui
GET /alerts/ui/count
```

Esto reduce errores y hace que listado y conteo usen los mismos criterios.

Usar joins para contexto

El frontend necesita contexto.

En vez de obligarlo a hacer varias llamadas, el backend ofrece respuestas preparadas.

Esto simplifica mucho la capa visual.

Limitar estados con `AlertStatus`

Usar:

```
Literal["open", "ack", "closed"]
```

evita estados inconsistentes.

Esto es especialmente importante porque el estado participa en la lógica de anti-duplicado.

Validar rangos de severidad

El backend comprueba que:

```
severity_min <= severity_max
```

Esto evita filtros incoherentes.

2 1 Ejemplo completo de flujo

1. Crear regla

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "Failed login auth",
  "enabled": true,
  "source": "auth",
  "severity_min": 3,
  "contains": "failed login"
}'
```

2. Enviar evento coincidente

```
curl -X POST http://localhost:8000/ingest \
-H "Content-Type: application/json" \
-d '{
  "source": "auth",
  "severity": 4,
  "message": "Failed login attempt for user admin",
}
```

```
"meta": {  
  "host": "server-01",  
  "user": "admin"  
}  
'
```

3. Consultar alertas para UI

```
curl http://localhost:8000/alerts/ui
```

4. Consultar detalle enriquecido

```
curl http://localhost:8000/alerts/1/ui
```

5. Reconocer alerta

```
curl -X PATCH http://localhost:8000/alerts/1 \  
-H "Content-Type: application/json" \  
-d '{  
  "status": "ack"  
}'
```

6. Cerrar alerta

```
curl -X PATCH http://localhost:8000/alerts/1 \  
-H "Content-Type: application/json" \  
-d '{  
  "status": "closed"  
}'
```

7. Consultar métricas

```
curl http://localhost:8000/metrics
```

2 2 Resumen técnico

La gestión de alertas representa la fase final del flujo SIEM del laboratorio.

Las alertas se generan en `/ingest`, pero se consultan y actualizan desde `/alerts`.

El schema `AlertOut` representa una alerta básica. El schema `AlertUIOut` añade información de regla y evento para facilitar el trabajo del frontend. El schema `AlertUpdate` permite modificar únicamente el estado de la alerta.

El flujo operativo queda representado por los estados:

```
open → ack → closed
```

Estos estados no solo afectan a la visualización, sino también a la lógica de anti-duplicado del motor de reglas.

La relación final del módulo es:

```
Rule + Event  
  ↓  
Alert  
  ↓  
AlertUIOut
```



Frontend / flujo SOC

Este módulo convierte las detecciones del backend en elementos gestionables por el usuario, acercando el proyecto a una herramienta básica de monitorización y análisis.